1. Overall Timeline (Now → Mid April)

You roughly have **5–6 weeks**.

Week	Milestone	Output
Week 1	HBM interface study + simplified protocol design	Interface spec
Week 2	RTL DUT model	Read/Write memory model
Week 3	Basic UVM environment	Agent + driver + monitor
Week 4	Scoreboard + reference model	Self checking
Week 5	Assertions + functional coverage	Protocol validation
Week 6	Neural network + final testing	ML + report

---

2. Phase-1: Understanding HBM Interface (Week 1)

You **DO NOT need full JEDEC HBM4**. That spec is massive.

Instead build a **simplified HBM-style interface**.

Basic Signals

Example interface:

clk
reset

cmd
addr
wdata
rdata

valid
ready
write_enable

Supported transactions

Just implement:

1. **READ**
2. **WRITE**
3. **BURST READ**
4. **BURST WRITE**

This is enough to demonstrate **HBM transaction verification**.

Output of Week-1

You should have:

• Interface diagram
• Transaction format
• Timing diagram

Example transaction:

WRITE
Address = 0x20
Data = 0xABCD

---

3. Phase-2: DUT Design (Week 2)

You need a **simple RTL memory controller**.

Architecture:

HBM Interface
    |
Controller Logic
    |
Memory Model

Memory can be:

reg [31:0] memory [0:1023]

Controller tasks:

• Accept read/write command
• Write into memory
• Return read data
• Handle burst transactions

Example operation:

WRITE addr 10 data 55
READ addr 10
→ output 55

This becomes the **DUT**.

---

4. Phase-3: UVM Environment (Week 3)

Now comes the **core part**.

Structure:

TEST
 |
ENV
 |

AGENT
 ├ Driver
 ├ Monitor
 ├ Sequencer
 |
Scoreboard
Coverage
Predictor

1️⃣ Sequence Item (Transaction)

Defines packet structure.

Example fields:

```
class mem_txn extends uvm_sequence_item;

rand bit [31:0] addr;
rand bit [31:0] data;
rand bit write;
rand int burst_len;

endclass
```

2️⃣ Sequencer

Generates random transactions.

Example:

```
WRITE addr random
READ addr random
BURST WRITE
BURST READ
```

3️⃣ Driver

Driver converts transaction → DUT signals.

Example:

```
@(posedge clk)
valid <= 1
addr <= txn.addr
data <= txn.data
write <= txn.write
```

4️⃣ Monitor

Monitor **captures DUT activity**.

It observes:

addr
data
cmd

and sends transaction to scoreboard.

---

5. Phase-4: Self Checking (Week 4)

This is the **most important part**.

Reference Model

You build a **golden memory model**.

Example:

ref_memory[address] = data

Whenever DUT writes:

update reference model

Whenever DUT reads:

compare DUT data with reference model

If mismatch:

UVM_ERROR

Scoreboard

Scoreboard compares:

DUT output
vs
Reference model output

If they match → PASS
If not → FAIL

---

6. Phase-5: Assertions + Coverage (Week 5)

Assertion Based Checking

Example checks:

• valid must not be high without ready
• read must not happen before command
• burst length must be valid

Example assertion idea:

valid |-> ready

Functional Coverage

Coverage ensures **all cases are tested**.

Example:

covergroup mem_cov;

coverpoint write;
coverpoint burst_len;

endgroup

Coverage ensures:

• read tested
• write tested
• burst tested

---

7. Phase-6: Neural Network Part (Week 6)

This part can be **separate from verification**.

Two easy ideas:

Option 1 (Recommended)

Use ML to **predict memory latency errors**.

Input features:

address
burst length
command type

Output:

expected latency

Train model using:

• Python
• MATLAB

Algorithms:

Random Forest
Neural Network

Option 2 (Better looking academically)

Use NN to detect **anomalous memory transactions**.

Dataset:

Generate simulation logs:

addr
cmd
burst
latency

Train NN to classify:

Normal
Faulty

9. Weekly Micro Timetable

Example weekly routine.

Weekdays

Time	Work
2 hrs	Reading / understanding
2 hrs	Implementation
1 hr	Debugging

Total:

5 hours/day

Weekend

Work
Integration
Simulation
Fixing errors

---

10. Final Deliverables

Your project should produce:

1️⃣ UVM Testbench

env
agent
driver
monitor
sequencer
scoreboard
coverage
assertions

2️⃣ Self Checking Verification

reference model
scoreboard
error detection

3️⃣ Simulation Results

Screenshots:

waveforms
coverage reports
assertion checks

4️⃣ Neural Network

Results:

training accuracy
prediction results
graphs